

TECHNOLOGY DOCUMENTATION

QR-Based Visitor Management System

A detailed breakdown of every technology used — what it is, where it is used, and why it was chosen

Field	Details
Document Type	Technology Documentation
System Name	QR-Based Visitor Management System
Version	v2.0.0
Prepared By	Amit Kushwaha
Department	MP Online Limited
Date	27 April 2026
Project Type	Internship Project

TECHNOLOGY STACK — QUICK REFERENCE

This table gives a bird's-eye view of every technology used in this project. Each one is explained in detail in the sections that follow.

Layer	Technology	Version	Used For
Backend	Python	3.11+	Core backend language — all server-side logic
	FastAPI	0.111.0	Web framework for building REST APIs
	Uvicorn	0.29.0	ASGI server that runs the FastAPI application
	Pydantic v2	2.7.1	Data validation for all API request/response models
	pydantic-settings	2.2.1	Loads configuration from .env file
Database	PostgreSQL	13+	Primary relational database for all data storage
	SQLAlchemy	2.0.30	ORM — maps Python classes to database tables
	psycopg2-binary	2.9.9	PostgreSQL database driver for Python
	Alembic	1.13.1	Database migration tool (installed, future use)
Auth	bcrypt	4.1.3	Password hashing with adaptive cost factor
	python-jose	3.3.0	JWT creation, signing, and verification
QR Code	qrcode	7.4.2	Generates QR code images from UUID tokens
	Pillow (PIL)	10.3.0	Image processing library used by qrcode to create PNG
Email	smtplib (stdlib)	built-in	Sends QR code via Gmail SMTP with STARTTLS
	email.mime (stdlib)	built-in	Constructs multipart HTML email with inline image
Utilities	python-dotenv	1.0.1	Loads .env file into environment variables
	python-multipart	0.0.9	Parses form-data (required for OAuth2 login form)
	aiofiles	23.2.1	Async file I/O (file upload support)
Frontend	React	18.2.0	Component-based UI framework
	React Router DOM	6.22.0	Client-side routing between pages
	Axios	1.6.7	HTTP client for calling the backend API
	html5-qrcode	2.3.8	Camera-based QR code reader in the browser
	react-scripts (CRA)	5.0.1	Build toolchain (webpack, babel, eslint)

Browser APIs	getUserMedia	Web API	Accesses device camera for photo capture
	Canvas API	Web API	Captures a video frame as an image
	FileReader API	Web API	Converts uploaded image files to base64
	localStorage	Web API	Stores JWT token and theme preference in browser
Testing	pytest	8.2.0	Backend testing framework
	httpx	0.27.0	Async HTTP client for testing FastAPI endpoints

1. BACKEND TECHNOLOGIES

1.1 Python 3.11

What is it?

Python is a high-level, general-purpose programming language known for its clean syntax and large ecosystem of libraries. It is the most widely used language for backend web development alongside JavaScript.

Where is it used in this project?

Python is the language of the entire backend. Every file inside backend/ is written in Python. This includes all API routes, business logic, database models, authentication, QR generation, and email sending.

Why Python?

- Fastest language for prototyping and building APIs — ideal for an internship timeline
- The qrcode, bcrypt, and smtplib libraries are all Python-native
- FastAPI (the web framework) requires Python 3.8+ and is built specifically for Python
- SQLAlchemy ORM works best with Python

Specific Python features used:

- Type hints (str | None, dict, Optional[int]) — required by FastAPI and Pydantic for automatic validation
- async/await — FastAPI runs on an async ASGI server; route handlers can be async
- dataclasses and enum — used for UserRole, VisitPurpose, VisitStatus enums in models.py
- f-strings — used throughout for readable string formatting
- @lru_cache — used on get_settings() in config.py so .env is read only once at startup

1.2 FastAPI (v0.111.0)

What is it?

FastAPI is a modern, high-performance Python web framework for building REST APIs. It is built on Starlette (the ASGI toolkit) and Pydantic (data validation). It automatically generates interactive API documentation (Swagger UI) at /docs.

Where is it used?

FastAPI is the web framework for the entire backend. Every API endpoint — login, visitor registration, QR scan, dashboard, user management — is defined as a FastAPI route function.

How it is used in this project:

FastAPI Feature	How it is used
APIRouter	Each feature group (auth, visitors, visits, admin) has its own router registered in main.py with a prefix like /api/auth
Depends()	Dependency injection — get_db() gives each route a database session; get_current_user() decodes the JWT; require_roles() checks user role
response_model=	Pydantic model used to validate and serialize the response. FastAPI strips any fields not in the schema.
HTTPException	Raised in routes and services to return structured error responses with correct HTTP status codes (401, 403, 404, 400)
OAuth2PasswordRequestForm	Used in the login route to accept form-urlencoded data (username + password) as required by OAuth2 standard
StaticFiles	app.mount('/uploads', StaticFiles(directory='uploads')) — serves visitor photos at /uploads/photos/{filename}
lifespan context	asynccontextmanager used to run startup code (create tables, seed admin) before the server starts accepting requests
CORSMiddleware	Added to allow the React frontend (localhost:3000) to make API calls to the backend (localhost:8000)

1.3 Uvicorn (v0.29.0)

What is it?

Uvicorn is an ASGI (Asynchronous Server Gateway Interface) web server. It is the process that actually listens on port 8000 and passes incoming HTTP requests to the FastAPI application.

Where is it used?

Uvicorn is not imported in code — it is the command used to start the server:

```
uvicorn main:app --reload --port 8000
```

What each part means:

- main — the Python file (main.py) that contains the FastAPI app
- app — the FastAPI() instance variable inside that file
- --reload — auto-restarts the server whenever any Python file changes (development only)
- --port 8000 — listens on port 8000

Why Uvicorn?

FastAPI is an ASGI framework which means it needs an ASGI server. Uvicorn is the standard recommended server for FastAPI. It supports async handlers and HTTP/1.1 efficiently.

1.4 Pydantic v2 (v2.7.1) + pydantic-settings (v2.2.1)

What is it?

Pydantic is a Python library for data validation using type annotations. You define a class with typed fields, and Pydantic automatically validates incoming data, rejects invalid input, and converts types. FastAPI uses Pydantic for all request and response models.

Where is it used?

Pydantic is used in two places in this project:

1. schemas/schemas.py — All request and response data models:

- VisitorRegister — validates the visitor registration form data
- TokenOut — shapes the login response (token, user_id, role, branch_id)
- UserCreate, UserOut — user creation and display
- VisitOut — full visit record response
- ChangePassword, ProfileUpdate — profile management

2. core/config.py — Application settings via pydantic-settings:

```
class Settings(BaseSettings):  
    DATABASE_URL: str = 'postgresql://...'  
    SECRET_KEY: str = 'change-this'  
    SMTP_ENABLED: bool = False
```

pydantic-settings reads each field from the .env file, validates the type, and makes it available as settings.DATABASE_URL, settings.SECRET_KEY, etc. The @lru_cache decorator on get_settings() ensures .env is parsed exactly once.

Key Pydantic features used:

- @field_validator — used on qr_hours to ensure value is between 1 and 8760
- Literal type — used for qr_validity_type to restrict to 5 valid string values
- Optional[EmailStr] — validates that email is a valid email format when provided
- from_attributes = True — allows Pydantic to read SQLAlchemy model objects directly

2. DATABASE TECHNOLOGIES

2.1 PostgreSQL

What is it?

PostgreSQL is a powerful, open-source relational database management system (RDBMS). It stores data in structured tables with rows and columns, supports complex queries, foreign keys, transactions, and many advanced data types.

Where is it used?

PostgreSQL is the primary database for the entire application. All data — visitor records, visit logs, user accounts, branches, audit logs — is stored in PostgreSQL running on localhost:5432, in a database named `qr_visitor_db`.

Tables in this project:

Table	What it stores
branches	Office locations — id, name, city, address, is_active, created_at
users	System users (guards, admins) — credentials, role, branch assignment, login history
visitors	Visitor records — name, phone (unique key), email, company, photo path
visits	Each visit event — QR token, expiry, check-in/out timestamps, duration, status, purpose
audit_logs	Tamper-evident log of every significant action — who did what, when, from which IP

Why PostgreSQL?

- Production-grade reliability — used at companies like Instagram, GitHub, Spotify
- Supports ENUMs natively — used for UserRole, VisitPurpose, VisitStatus
- ACID compliant — data is never partially written (either all of a visit is saved or none)
- Excellent performance with indexed columns — `qr_token` and `email` are both indexed for fast lookup
- Works perfectly with SQLAlchemy ORM

2.2 SQLAlchemy (v2.0.30)

What is it?

SQLAlchemy is a Python ORM (Object-Relational Mapper). It lets you define database tables as Python classes and write database queries using Python instead of raw SQL. Version 2.0 is a major rewrite with better type support.

Where is it used?

SQLAlchemy is used in:

- app/db/models.py — ORM class definitions for all 5 tables
- app/db/session.py — database engine and session factory
- app/db/init_db.py — create_all() to create tables, ensure_columns() for auto-migration
- All service files — db.query(), db.add(), db.commit(), db.flush(), db.refresh()

How SQLAlchemy works in this project:

```
# Define a table as a Python class (models.py)
class Visit(Base):
    __tablename__ = 'visits'
    id = Column(Integer, primary_key=True)
    qr_token = Column(String(36), unique=True, index=True)
    status = Column(SAEnum(VisitStatus))

# Query in a service (no SQL needed)
visit = db.query(Visit).filter(Visit.qr_token == token).first()
```

Key SQLAlchemy features used:

- relationship() — defines foreign key links (Visit.visitor, Visit.branch) for easy navigation
- back_populates — bidirectional relationship (Visitor.visits, Visit.visitor)
- declarative_base() — base class for all ORM models
- SAEnum — maps Python enum classes to PostgreSQL ENUM types
- func.avg(), func.count(), func.date() — SQL aggregation functions for dashboard queries
- pool_size=10, max_overflow=20 — connection pool handles 30 simultaneous DB connections

2.3 psycopg2-binary (v2.9.9)

What is it?

psycopg2 is the PostgreSQL database adapter for Python. It is the actual library that connects Python to PostgreSQL and sends SQL queries to the database. SQLAlchemy uses it under the hood.

The -binary suffix means it comes with pre-compiled PostgreSQL libraries bundled in — no separate PostgreSQL client installation needed. It is listed in requirements.txt and works automatically once installed. Developers never call it directly.

It is used in the DATABASE_URL connection string:

```
DATABASE_URL = postgresql://postgres:password@localhost:5432/qr_visitor_db
```

The postgresql:// prefix tells SQLAlchemy to use the psycopg2 driver.

3. AUTHENTICATION TECHNOLOGIES

3.1 bcrypt (v4.1.3)

What is it?

bcrypt is a password hashing function. Unlike encryption (which is reversible), hashing is one-way — you cannot reverse a bcrypt hash back to the original password. It is specifically designed to be slow, making brute-force attacks impractical.

Where is it used?

bcrypt is used in `app/core/security.py` for two functions:

```
import bcrypt

def hash_password(password: str) -> str:
    salt = bcrypt.gensalt(rounds=12)      # generates random salt
    return bcrypt.hashpw(password.encode(), salt).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return bcrypt.checkpw(plain.encode(), hashed.encode())
```

When is it called?

- `hash_password()` — called in `user_service.py` when creating a new user or changing a password
- `verify_password()` — called in `auth.py` login route to compare the submitted password against the stored hash

Why bcrypt directly (not passlib)?

The original code used `passlib[bcrypt]`. `passlib` tries to read `bcrypt.__about__.__version__` at import time. `bcrypt v4.x` removed the `__about__` module, causing a crash: 'password cannot be longer than 72 bytes'. The fix was to call `bcrypt` directly — one less library, zero version-conflict risk.

What is rounds=12?

The cost factor 12 means `bcrypt` performs $2^{12} = 4,096$ rounds of computation per hash. This means hashing one password takes ~150ms — fast enough for login but impossibly slow for an attacker trying millions of passwords per second.

3.2 python-jose (v3.3.0) — JWT Tokens

What is it?

`python-jose` is a Python library for creating and verifying JSON Web Tokens (JWT). A JWT is a compact, signed string that proves a user's identity without the server needing to store session data.

Where is it used?

python-jose is used in app/core/security.py:

```
from jose import JWTError, jwt

def create_access_token(data: dict) -> str:
    payload = data.copy()
    payload['exp'] = datetime.utcnow() + timedelta(minutes=480)
    return jwt.encode(payload, SECRET_KEY, algorithm='HS256')

def decode_token(token: str) -> Optional[dict]:
    return jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
```

JWT Flow in this project — step by step:

Step	What happens
1	User submits email + password to POST /api/auth/login
2	Backend finds user by email, calls verify_password() with bcrypt
3	If valid: calls create_access_token({ sub: email }) — signs a JWT with SECRET_KEY using HS256
4	JWT is returned to the frontend as { access_token: 'eyJ...' }
5	Frontend stores the token in localStorage and sends it in every request: Authorization: Bearer eyJ...
6	Backend dependency get_current_user() calls decode_token() — if valid, returns the User object
7	If token is expired or invalid: JWTError is caught, HTTP 401 is returned, frontend redirects to /login

What is HS256?

HS256 (HMAC-SHA256) is the signing algorithm. The token is signed using a shared SECRET_KEY known only to the server. Anyone can read the JWT payload (it is base64 encoded, not encrypted), but they cannot forge a valid signature without the SECRET_KEY.

4. QR CODE TECHNOLOGIES

4.1 uuid (Python standard library)

What is it?

uuid is a Python built-in module for generating Universally Unique Identifiers — 128-bit random numbers formatted as a string like: fc48f886-9fc3-4e1f-8282-800998f58dbc. No two UUID v4 values are ever the same.

Where is it used?

```
import uuid

def generate_token() -> str:
    return str(uuid.uuid4()) # generates a new random UUID every call
```

How it is used in the QR flow:

- When a visitor registers, generate_token() is called to create a unique token
- This token is stored in the visits.qr_token column (UNIQUE constraint)
- The same token is embedded in the QR code image
- When the guard scans the QR, the token is extracted and looked up in the database

Why UUID v4?

UUID v4 uses cryptographic randomness (2^{122} possible values). This makes it impossible to guess or predict a valid QR token. Even if an attacker knew the format, they would need billions of years to find a valid token by brute force.

4.2 qrcode library (v7.4.2)

What is it?

qrcode is a Python library that generates QR code images. You give it a string (the UUID token), and it creates a black-and-white QR code image that any camera can scan and decode back to the original string.

Where is it used?

```
import qrcode, base64
from io import BytesIO

def build_qr_image(token: str) -> str:
    qr = qrcode.QRCode(
        version=1, # smallest size
        error_correction=qrcode.constants.ERROR_CORRECT_H, # 30% damage OK
        box_size=10, # pixels per module
        border=4, # quiet zone
```

```

)
qr.add_data(token)
qr.make(fit=True)
img = qr.make_image(fill_color='black', back_color='white')
buf = BytesIO()
img.save(buf, format='PNG')
b64 = base64.b64encode(buf.getvalue()).decode()
return f'data:image/png;base64,{b64}'

```

What each setting means:

Parameter	Meaning
version=1	QR version determines size. Version 1 = 21x21 modules. fit=True auto-selects the right version for the data.
ERROR_CORRECT_H	30% error correction — the QR can still be read even if 30% of it is damaged or obscured (e.g. phone screen glare).
box_size=10	Each QR module (black/white square) is 10 pixels wide. Determines image resolution.
border=4	4-module white border around the QR code — required by the QR specification for reliable scanning.
data:image/png;base64,...	The QR image is saved to memory (not disk), base64-encoded, and returned as a data URI. The frontend displays it directly in an tag.

4.3 Pillow (PIL — v10.3.0)

What is it?

Pillow is the Python Imaging Library — a powerful library for opening, manipulating, and saving image files. The qrcode library uses Pillow internally to create the PNG image.

Pillow is not imported directly in this project's code. It is a dependency of the qrcode library. When `qr.make_image()` is called and `img.save(buf, format='PNG')` runs, Pillow performs the actual image encoding. Without Pillow installed, qrcode would throw an `ImportError`.

4.4 html5-qrcode (v2.3.8) — Frontend QR Scanner

What is it?

html5-qrcode is a JavaScript library that accesses the device camera through the browser and scans QR codes in real time. It works entirely in the browser — no server processing is needed for scanning.

Where is it used?

html5-qrcode is used in `ScannerPage.js`:

```
import { Html5QrcodeScanner } from 'html5-qrcode';
```

```
const scanner = new Html5QrcodeScanner(  
  'qr-reader', // ID of the div to render camera into  
  { fps: 10, qrbox: 250 }, // scan 10 frames/sec, 250px viewfinder  
  false // verbose = false  
);  
scanner.render(onSuccess, onError);
```

How the scanning works:

- The library accesses the device camera using the browser's getUserMedia API
- It captures 10 frames per second from the camera video stream
- Each frame is analysed using a QR decoding algorithm (ZXing under the hood)
- When a QR code is found, the decoded text (the UUID token) is passed to the onSuccess callback
- The frontend then calls POST /api/visits/scan/{token} with that token
- A 3-second cooldown prevents the same QR from triggering double scans

5. EMAIL TECHNOLOGY

5.1 smtplib (Python standard library)

What is it?

smtplib is Python's built-in library for sending emails using the SMTP protocol. SMTP (Simple Mail Transfer Protocol) is the standard internet protocol for sending email between servers.

Where is it used?

smtplib is used in app/utils/email.py to connect to Gmail and send the QR code email:

```
import smtplib

with smtplib.SMTP('smtp.gmail.com', 587) as server:
    server.ehlo()          # identify ourselves to the server
    server.starttls()      # upgrade to encrypted TLS connection
    server.login(SMTP_USER, SMTP_PASSWORD)  # authenticate
    server.sendmail(FROM, to_email, msg.as_string())
```

Why Gmail SMTP with STARTTLS on port 587?

- Port 587 is the standard SMTP submission port — requires authentication
- STARTTLS upgrades a plain connection to TLS encryption — credentials are never sent in plaintext
- Gmail is a reliable free SMTP server — no dedicated email server needed for this project
- App Passwords bypass 2FA — a 16-digit application-specific password avoids storing the main account password

5.2 email.mime (Python standard library)

What is it?

email.mime is Python's built-in module for constructing MIME (Multipurpose Internet Mail Extensions) email messages. MIME allows emails to contain HTML, attachments, and inline images.

Where is it used and why each part?

Class used	Purpose in this project
MIMEMultipart('related')	Creates a multipart container that allows mixing HTML text and inline images in one email. 'related' type tells email clients the image is referenced within the HTML.
MIMEText(html_body, 'html')	Attaches the HTML body of the email — contains the greeting, QR instructions, and the img tag referencing the QR code.
MIMEImage(qr_bytes, _subtype='png')	Attaches the QR code PNG image bytes. Content-ID header 'qr_code' lets the HTML reference it via src='cid:qr_code'.

The complete email structure built by these classes:

```
MIMEMultipart('related')
├── MIMEText (HTML body with <img src='cid:qr_code'> tag)
└── MIMEImage (QR code PNG, Content-ID: qr_code, inline disposition)
```

6. FRONTEND TECHNOLOGIES

6.1 React (v18.2.0)

What is it?

React is a JavaScript library for building user interfaces. Instead of manipulating HTML directly, you write components — reusable pieces of UI that automatically update when their data changes.

Where is it used?

React is the complete frontend framework. Every page and UI element is a React component. The entire src/ folder is React code.

React concepts used in this project:

Concept	How it is used
useState	Local component state — form inputs, loading flags, scan results, modal open/close
useEffect	Fetch data when a page loads (dashboard, branches list), set timers (QR countdown), cleanup on unmount (stop camera)
useRef	Direct DOM access — videoRef for camera stream, canvasRef for photo capture, fileInputRef for file picker
useCallback	Memoises stopCamera() so it is stable across renders and safe to use in useEffect cleanup
useContext	useAuth() reads AuthContext (logged-in user), useTheme() reads ThemeContext (dark/light mode)
createContext / Provider	AuthContext and ThemeContext share state globally across all components without prop drilling
Custom Hooks	useToast() — encapsulates toast notification state with show/hide/auto-dismiss logic
Conditional rendering	{mode === 'camera' && <video />} — show camera only when active, show different tabs on dashboard
Event handlers	onClick, onChange, onSubmit — all form interactions and button actions

6.2 React Router DOM (v6.22.0)

What is it?

React Router is a library that enables client-side routing in React applications. It allows the URL to change and different components to render without the browser reloading the entire page.

Where is it used?

React Router is configured in App.js and used across all pages:

Component / Hook	Used for
BrowserRouter	Wraps the entire app — enables history-based URL routing
Routes + Route	Maps URL paths to page components — e.g. /scan → ScannerPage, /dashboard → DashboardPage
Navigate	Programmatic redirect — guards go to /scan on login, others go to /dashboard; unauthorised users go to /login
useNavigate()	Navigate after form submit — after registration navigate('/qr-display', { state: { visit } }) passes QR data
useLocation()	QRDisplayPage reads location.state.visit to get the registered visit data passed from VisitorFormPage
Link	Navigation links in Navbar that change URL without page reload

6.3 Axios (v1.6.7)

What is it?

Axios is a JavaScript HTTP client library. It makes it easy to send HTTP requests (GET, POST, PUT, DELETE) to the backend API and handle responses and errors.

Where is it used?

Axios is configured once in src/utils/api.js and used from every page:

```
const api = axios.create({
  baseURL: 'http://localhost:8000/api',
  headers: { 'Content-Type': 'application/json' },
});
```

Two interceptors do the heavy lifting:

Request interceptor — automatically attaches the JWT token to every request:

```
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Response interceptor — handles expired sessions automatically:

```
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
  }
);
```

> ;

API groupings (from api.js):

- authAPI — login, me, updateProfile, changePassword
- visitorAPI — register, list, getById
- visitAPI — scan, list
- adminAPI — dashboard, report, auditLogs, branches, createBranch, users, createUser, deleteUser

6.4 react-scripts / Create React App (v5.0.1)

What is it?

react-scripts is the build toolchain for Create React App (CRA). It wraps webpack, Babel, ESLint, and Jest into a single package so developers can start coding without configuring a build system.

What each tool inside react-scripts does:

Tool	What it does
webpack	Bundles all JavaScript files (and their imports) into a single optimised output file for the browser
Babel	Transpiles modern JavaScript (ES2022+, JSX) to older syntax that all browsers understand
ESLint	Code linter — warns about unused variables, missing dependencies in useEffect, etc.
source-map-loader	Loads source maps for debugging. Caused 23 warnings for html5-qrcode because its source .ts files were missing — fixed with GENERATE_SOURCEMAP=false in frontend/.env

Two commands are available:

- npm start — starts webpack dev server at localhost:3000 with hot-reload
- npm run build — creates an optimised production build in the build/ folder

7. BROWSER WEB APIS

These are not libraries you install — they are built-in capabilities provided by modern browsers. The project uses four of them.

7.1 MediaDevices.getUserMedia (Camera Access)

What is it?

`getUserMedia` is a Web API that requests access to the user's camera (or microphone). When called, the browser shows a permission prompt. If granted, it returns a `MediaStream` object representing the live camera feed.

Where is it used?

Used in `CameraCapture.js` (visitor photo) and indirectly by `html5-qrcode` (QR scanning):

```
const stream = await navigator.mediaDevices.getUserMedia({
  video: { facingMode: 'user', width: 320, height: 240 }
});
videoRef.current.srcObject = stream; // show live feed in <video> element
```

How it is used for visitor photo capture:

- `facingMode: 'user'` selects the front-facing camera (selfie camera on mobile)
- The stream is assigned to a `<video>` element's `srcObject` property
- When the user clicks Capture, a frame is drawn to a Canvas
- When done, `stream.getTracks().forEach(t => t.stop())` releases the camera

7.2 Canvas API (Photo Capture)

What is it?

The Canvas API is a browser API that lets JavaScript draw graphics onto an HTML `<canvas>` element. It can also read pixel data from video frames, making it the standard way to capture a still image from a live video stream.

Where is it used?

```
const capturePhoto = () => {
  canvas.width = video.videoWidth;
  canvas.height = video.videoHeight;
  canvas.getContext('2d').drawImage(video, 0, 0); // draw current frame
  const dataUrl = canvas.toDataURL('image/jpeg', 0.85); // to base64
  onCapture(dataUrl); // send to parent component
};
```

`drawImage(video, 0, 0)` copies the current video frame onto the canvas. `toDataURL()` converts that canvas image to a base64 JPEG string that is sent to the backend.

7.3 FileReader API (File Upload)

What is it?

FileReader is a browser API that reads the contents of File objects (from file input elements) asynchronously. It can convert a file to a base64 data URL, which is exactly what the backend expects for visitor photos.

Where is it used?

```
const reader = new FileReader();
reader.onload = (ev) => {
  const dataUrl = ev.target.result; // 'data:image/jpeg;base64,...'
  onCapture(dataUrl);
};
reader.readAsDataURL(file); // triggers async read
```

This is the file upload alternative to camera capture in CameraCapture.js. Both paths produce the same output — a base64 data URI — so the backend handles them identically.

7.4 localStorage (Session Persistence)

What is it?

localStorage is a browser API that stores key-value string data persistently in the browser. Unlike cookies, localStorage is not sent to the server. Data survives page refreshes and browser restarts.

What is stored and why:

Key	Value	Why
token	JWT access token	Attached to every API request via Axios interceptor. Cleared on logout or 401 response.
user	JSON user object	Stores user_name, user_role, branch_id. Used to show role-appropriate nav links and redirect on login.
vms-theme	'dark' or 'light'	Remembers the user's preferred theme across sessions. Read on page load in ThemeContext.

8. UTILITY LIBRARIES

8.1 python-dotenv (v1.0.1)

What is it?

python-dotenv loads environment variables from a `.env` file into Python's `os.environ`. It allows sensitive configuration (passwords, secret keys, database URLs) to be stored in a file that is never committed to version control.

Where is it used?

python-dotenv is used indirectly through pydantic-settings. The Settings class in `config.py` has `env_file = '.env'` in its Config — pydantic-settings uses python-dotenv under the hood to read the file. The result is that `DATABASE_URL`, `SECRET_KEY`, `SMTP_PASSWORD`, etc. are loaded automatically from `backend/.env` on startup.

What is in the .env file:

- `DATABASE_URL` — PostgreSQL connection string with username and password
- `SECRET_KEY` — random string used to sign JWTs
- `SMTP_USER`, `SMTP_PASSWORD` — Gmail credentials for email sending
- `QR_EXPIRY_HOURS` — how long QR codes are valid (default 24)
- `SMTP_ENABLED` — True/False toggle for email sending

8.2 python-multipart (v0.0.9)

What is it?

python-multipart is a Python library for parsing multipart form data — the format used when submitting HTML forms with file uploads, or when using OAuth2 form-based login.

In this project, python-multipart is required for the FastAPI `OAuth2PasswordRequestForm` dependency in the login route. When a frontend sends the login credentials as `application/x-www-form-urlencoded` (which is the OAuth2 standard), FastAPI needs python-multipart to parse that body format. Without it, FastAPI raises an `ImportError` at startup.

8.3 aiofiles (v23.2.1)

What is it?

aiofiles is a Python library that provides async file I/O operations. It allows reading and writing files without blocking the async event loop — important for keeping the server responsive during file operations.

aiofiles is installed and available for async file operations when needed. In the current visitor photo saving code, the file is written synchronously (with `open()`), which is acceptable for the small photo files in this project. aiofiles would be used in a production scenario where many photos are uploaded simultaneously.

9. TESTING TECHNOLOGIES

9.1 pytest (v8.2.0)

What is it?

pytest is the most popular Python testing framework. It discovers test files automatically (`test_*.py`), provides clear output on failures, and supports fixtures for setup/teardown.

pytest is installed in `requirements.txt` for testing backend API routes and services. Test files would be placed in the `backend/tests/` directory and named `test_*.py`. pytest is run with the command: `pytest` from the `backend/` directory.

9.2 httpx (v0.27.0)

What is it?

httpx is an async-capable Python HTTP client. FastAPI's testing documentation recommends httpx as the test client for testing FastAPI endpoints without starting a real server.

httpx is used with FastAPI's `TestClient` to make test requests to the API. It simulates real HTTP requests (including headers, body, auth) against the app in memory, allowing endpoints to be tested without running uvicorn.

10. CSS CUSTOM PROPERTIES (VARIABLES) — THEME SYSTEM

What is it?

CSS Custom Properties (also called CSS Variables) are values defined in CSS that can be reused throughout a stylesheet. They are declared with `--` prefix (e.g. `--bg`, `--text`) and referenced with `var()`. They can be changed dynamically with JavaScript.

How the dark/light theme works in this project:

All colours in `index.css` are defined as CSS variables under `:root` (dark mode, the default) and overridden under `[data-theme='light']`:

```
:root {
  --bg:      #0f172a; /* dark navy background */
  --text:    #f1f5f9; /* light text */
  --primary: #6366f1; /* indigo accent */
}

[data-theme='light'] {
  --bg:      #f1f5f9; /* light background */
  --text:    #0f172a; /* dark text */
  --primary: #4f46e5; /* slightly darker accent */
}
```

`ThemeContext.js` changes the theme by setting a single attribute on `document.body`:

```
document.body.setAttribute('data-theme', theme); // 'dark' or 'light'
```

This single attribute change triggers the CSS override — every `var(--bg)`, `var(--text)` etc. across all 700+ lines of CSS updates instantly with zero JavaScript or React re-renders. This is why theme switching is instantaneous.

11. COMPLETE VERSION REFERENCE

This table lists every library and its exact version as it appears in requirements.txt (backend) and package.json (frontend).

11.1 Backend (requirements.txt)

Package	Version	Purpose
fastapi	0.111.0	Web framework — defines all API routes
uvicorn[standard]	0.29.0	ASGI web server — runs the FastAPI app
sqlalchemy	2.0.30	ORM — maps Python classes to database tables
psycopg2-binary	2.9.9	PostgreSQL driver for Python
alembic	1.13.1	Database migration tool
qrcode	7.4.2	Generates QR code PNG images
pillow	10.3.0	Image processing (dependency of qrcode)
python-jose[cryptography]	3.3.0	JWT creation and verification
bcrypt	4.1.3	Password hashing
python-dotenv	1.0.1	Loads .env file
python-multipart	0.0.9	Parses form-data for OAuth2 login
pydantic[email]	2.7.1	Data validation + email format validator
pydantic-settings	2.2.1	Loads settings from .env into Pydantic model
aiofiles	23.2.1	Async file I/O
pytest	8.2.0	Testing framework
httpx	0.27.0	Async HTTP client for testing FastAPI

11.2 Frontend (package.json)

Package	Version	Purpose
react	^18.2.0	Core UI library
react-dom	^18.2.0	Renders React components into the browser DOM
react-router-dom	^6.22.0	Client-side URL routing
axios	^1.6.7	HTTP client for API calls
html5-qrcode	^2.3.8	Camera-based QR code scanner
react-scripts	5.0.1	Build toolchain (webpack, Babel, ESLint)

qrcode.react	^3.1.0	React QR code display component (installed, available for use)
react-qr-scanner	^1.0.0-alpha.11	Alternative QR scanner (installed, html5-qrcode used instead)

— End of Technology Documentation —

QR-Based Visitor Management System | Technology Documentation | v2.0.0 | April 2026